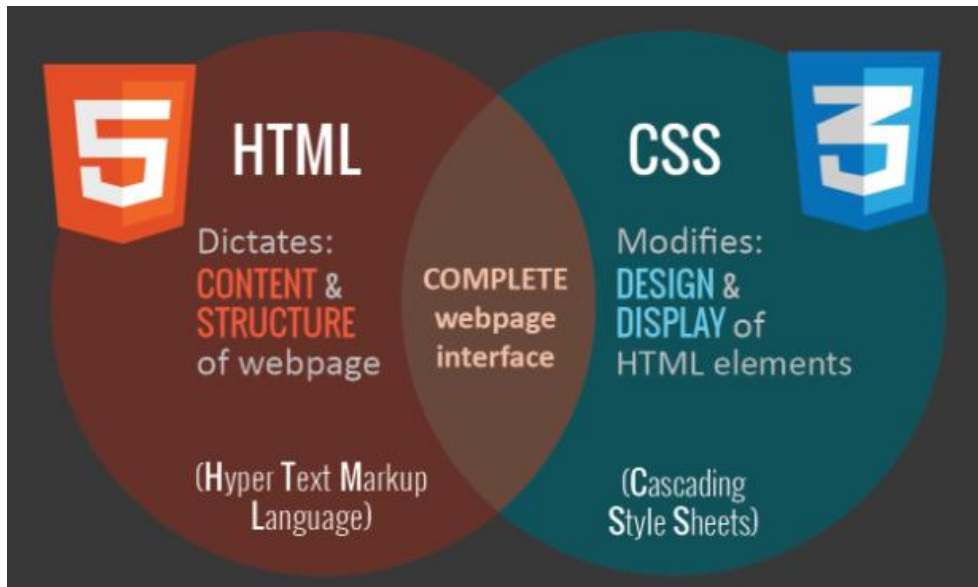


CSS

CSS (**Cascading Style Sheets**) is a fundamental language for styling web pages. Using CSS, you can control exactly how HTML elements look in the browser, presenting your documents to your users with whatever design and layout you like. It dictates how HTML elements should be displayed on the screen, controlling their appearance, layout, and visual presentation. This separation of content (HTML) and presentation (CSS) enhances code organization, maintainability, and reusability.



Benefits of Using CSS:

- ✓ **Separation of Concerns:** Keeps the structure (HTML) and presentation (CSS) separate, making your code cleaner, more maintainable, and easier to update.
- ✓ **Reusability:** Define styles once and apply them to multiple elements throughout your website.
- ✓ **Cross-browser Compatibility:** CSS helps ensure consistent styling across different web browsers.
- ✓ **Improved Accessibility:** CSS can be used to improve the accessibility of your website for users with disabilities.

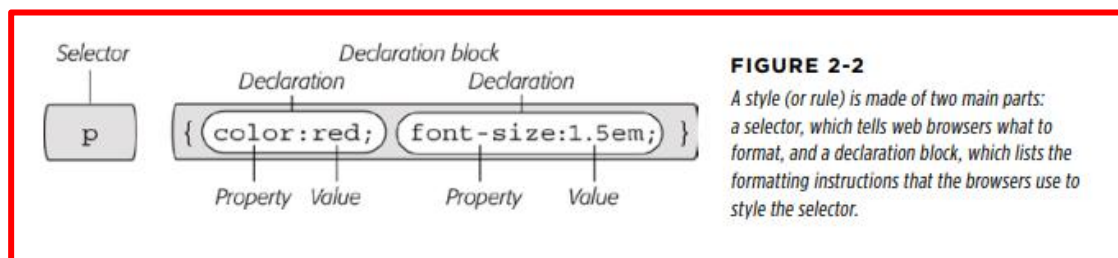
CSS Syntax

CSS is a rule-based language — you define rules by specifying groups of styles that should be applied to particular elements or groups of elements on your web page. CSS rules follow a specific structure that dictates how HTML elements are styled. CSS syntax consists of two main parts: **selectors and Declaration Block**.

Syntax:

```
selector {  
  property: value;  
}
```

- ✓ **Selector:** This is the HTML element you want to style. It could be an element like p, h1, div, or a class or id.
- ✓ **Declaration Block:** The code following the selector includes all the formatting options you want to apply to the selector. The block begins with an opening brace ({) and ends with a closing brace (}). Each declaration block can contain one or more declarations. Inside each declaration, there are one or more property and values separated by colon.
- ❖ **Declaration.** Between the opening and closing braces of a declaration block, you add one or more declarations, or formatting instructions. Every declaration has two parts, a property and a value. A colon separates the property name and its value, and the whole declaration ends with a semicolon.
- ❖ **Property:** This defines what aspect of the element you want to change, such as color, font-size, background, etc.
- ❖ **Value:** This specifies the value you want to apply to the property, such as a color (red), size (16px), or a type of font (Arial).



Example:

```
/* Selector: Targets all paragraph elements */
p {
  /* Declaration Block */
  color: blue;
  font-size: 1.5em;
}
```

This rule styles all <p> elements with blue text and a font size of 1.5 times bigger than the font size of its parent.

Inserting CSS

There are three different ways to apply CSS to an HTML document

- ✓ external stylesheets
- ✓ internal stylesheets,
- ✓ inline styles.

Inline styles/ Inline css

You can apply styles directly to an HTML element using the style attribute. The style attribute is used to apply styles to that specific element. Inside the quotes, you list CSS properties and their values, separated by semicolons.

This method is quick but not recommended for larger projects, as it mixes HTML and CSS. Since HTML and CSS are mixed, they are harder to maintain.

Example:

```
<p style="color: red; font-size: 16px;">This is a red text.</p>
```

This is the inline CSS that applies directly to the <p> element with red text with font size 16 pixels.

Internal styles/Internal CSS

Internal CSS is used to apply CSS on a single document or page. It can affect all the elements of the page. It is written inside the style tag within head section of html. It is suitable for smaller styles or when you want to keep CSS within the same HTML file.

For sites with more than one page, internal stylesheets are less efficient. To apply uniform CSS styling to multiple pages using internal stylesheets, you must repeat the internal stylesheet across every web page. The efficiency penalty carries over to site maintenance too. With CSS in internal stylesheets, there is the risk that even one simple styling change may require edits to multiple web pages.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
  <style>
    p {
      color: blue;
    }
  </style>
</head>
<body>
  <p>This is a paragraph.</p>
</body>
</html>
```

Output:

This is a paragraph.

External stylesheet/ External CSS

An external stylesheet contains CSS in a separate file with a .css extension. This is the most common and useful method of bringing CSS to a document. This is the most efficient way to manage styles for HTML pages because it allows you to link the same CSS file to multiple HTML files and multiple CSS files in single HTML file. It uses the <link> tag on every pages and the <link> tag should be put inside the head section.

```
<link rel="stylesheet" href="styles.css" />
```

This <link> element tells the browser that we have a stylesheet using the rel attribute, and the location of that stylesheet as the value of the href attribute.

We can link the stylesheets from the file system as well as cdn (Content Delivery Network).

```
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
```

Example:

Style.css

```
body {
  background-color: lightgray;
}

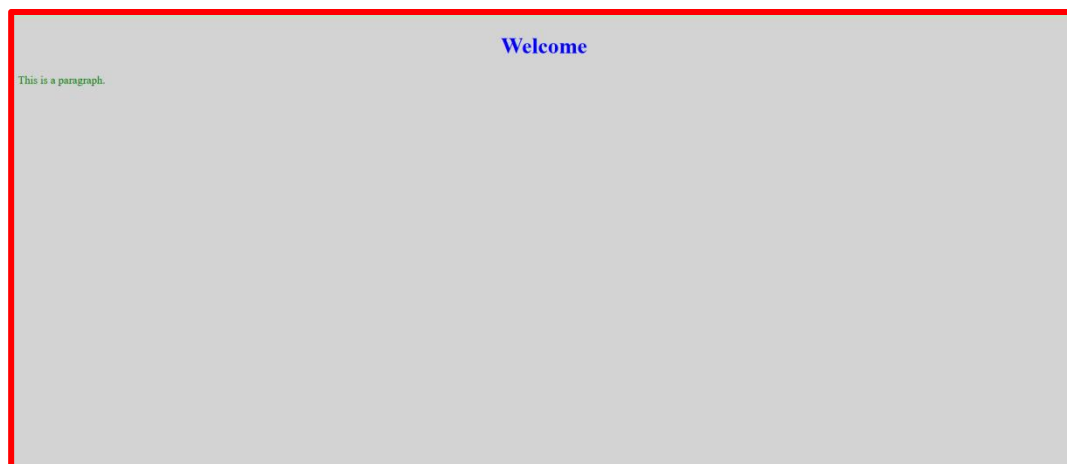
h1 {
  color: blue;
  text-align: center;
}

p {
  color: green;
  font-size: 16px;
}
```

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

Output:



CSS Selectors

In CSS, a selector is used to target and style specific HTML elements on a web page. Selectors define which elements the rules inside the CSS block should apply to. There are different types of selectors in CSS. They are

- ✓ Universal Selector (*)
- ✓ CSS Group Selector
- ✓ CSS Element/tag Selector
- ✓ **CSS Class Selector (.)**
- ✓ **CSS Id Selector (#)**
- ✓ Attribute Selector
- ✓ Descendent Selector (space)
- ✓ Child Selector (>)
- ✓ Pseudo Classes (:)
- ✓ Combinators

Universal Selector

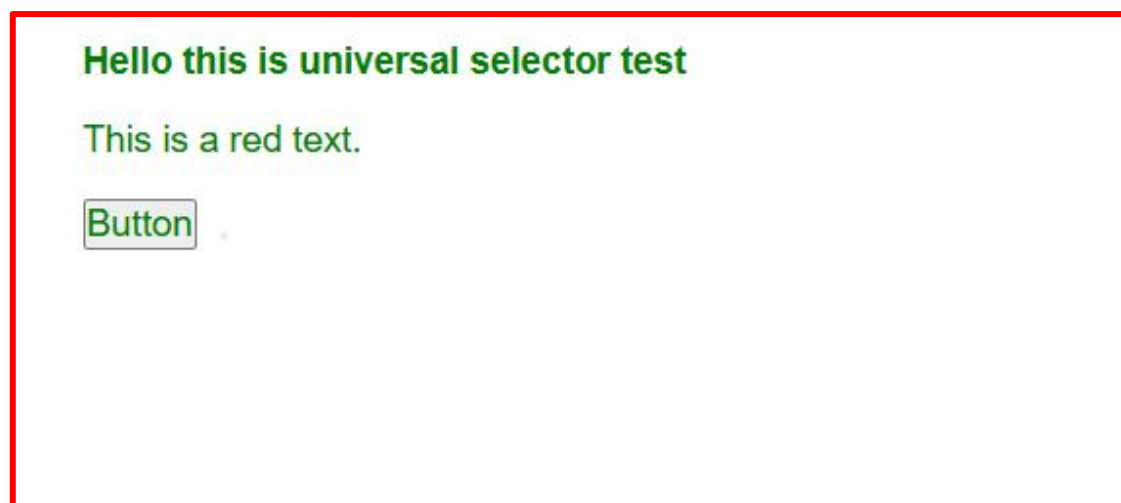
The universal selector is used as a wildcard character (*). It selects all the elements on the pages. The most common use is to create a CSS reset. This involves applying default styles to all elements to ensure consistent behavior across different browsers.

The universal selector can be useful for applying styles globally across your website, but use it carefully, as it affects all elements.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
  <style>
    * {
      Margin-left: 10px;
      padding: 0;
      font-family: Arial, sans-serif;
      font-size: 15px;
      color: green;
    }
  </style>
</head>
<body>
  <h1>Hello this is universal selector test</h1>
  <p>This is a red text.</p>
  <button type="button">Button</button>
</body>
</html>
```

Output:



In the above example, all the text will have same font family as Arial, sans-serif with font size 15 px with color green.

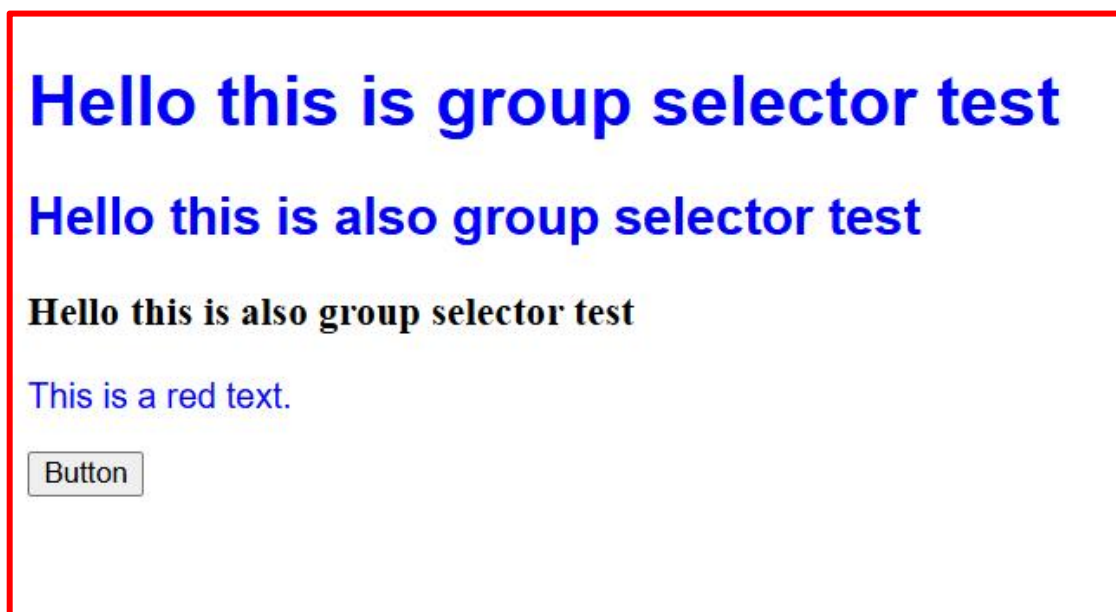
Group Selector

A group selector in CSS allows you to apply the same styles to multiple elements by grouping them together. This is done by separating the selectors with a comma.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
  <style>
    h1, h2, p {
      color: blue;
      font-family: Arial, sans-serif;
    }
  </style>
</head>
<body>
  <h1>Hello this is group selector test</h1>
  <h2>Hello this is also group selector test</h2>
  <h3>Hello this is also group selector test</h3>
  <p>This is a red text.</p>
  <button type="button">Button</button>
</body>
</html>
```

Output:



In the above example, you can see h1, h2, p are grouped and for them font family and font color are set same. Except than this group, other elements are not affected.

Element/Tag Selector

Element selectors are the foundation of CSS styling. They provide a straightforward way to target and style specific HTML elements based on their tag names, allowing you to quickly and easily apply consistent styles to your web pages. With them, you can make sweeping design changes to a page with very little effort.

For example, when you want to format every paragraph of text on a page, using the same font, color, and size, you merely create a style using p (as in the <p> tag) as the selector. In essence, a tag selector redefines how a browser displays a particular tag. Tag selectors are easy to spot in a CSS rule, since they bear the exact same name as the tag they style—p, h1, table, img, and so on.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tag Selector with Internal CSS</title>
  <style>
    /* Tag Selector */
    h1 {
      color: darkred;
      text-align: center;
    }

    p {
      color: green;
      font-family: 'Arial', sans-serif;
      line-height: 1.6;
    }

    div {
      background-color: lightgray;
      padding: 20px;
      border-radius: 8px;
    }
  </style>
</head>
<body>

  <h1>Welcome to My Web Page!</h1>

  <div>
    <p>This is a paragraph inside a div container.</p>
    <p>Another paragraph inside the div.</p>
  </div>

</body>
</html>
```


Output



CSS Class Selector (.)

A class selector in HTML allows you to target and style HTML elements that have a specific class attribute. The class selector is used in CSS to apply styles to elements with a matching class name. Unlike tag selectors that apply styles to all elements of a specific type, class selectors can apply styles to specific elements, even if they are different types (e.g., <div>, <p>, <h1>, etc.), as long as they share the same class. It's a powerful tool for applying styles to multiple elements that share a common characteristic or purpose, without having to style each element individually.

Process of using class selector.

1. In HTML, you apply a class to an element using the class attribute. An element can have multiple classes, separated by spaces.
2. In CSS, you use a period (.) followed by the class name to create a class selector.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Class Selector Example 2</title>
  <style>
    /* Class Selector 1 */
    .card {
      border: 2px solid #333;
      border-radius: 8px;
      padding: 15px;
      margin: 20px;
      background-color: #f4f4f4;
    }

    /* Class Selector 2 */
    .button {
      background-color: #4CAF50;
      color: white;
      padding: 10px 20px;
      font-size: 16px;
    }

    /* Class Selector 3 */
    .warning {
      color: #d9534f;
      font-weight: bold;
      font-size: 18px;
    }
  </style>
</head>
<body>
  <div class="card">
    <p>This is a paragraph inside a div container.</p>
    <p>Another paragraph inside the div.</p>
  </div>
  <button class="button">Click Me</button>
  <div class="warning">Warning!</div>
</body>
</html>
```

```

    </style>
</head>
<body>

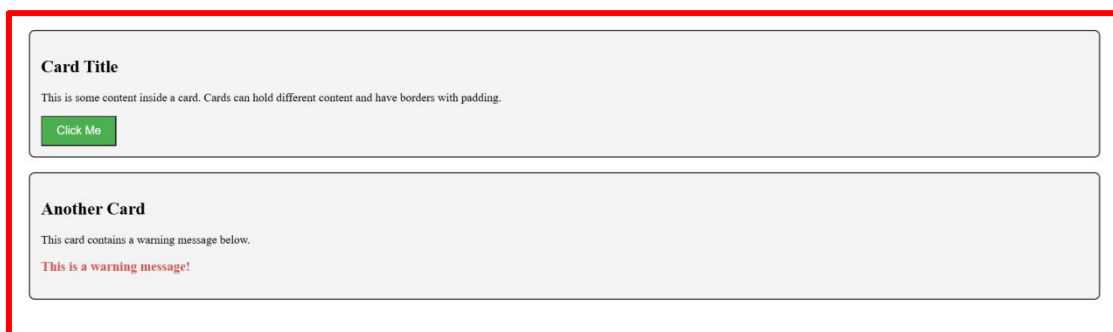
    <div class="card">
        <h2>Card Title</h2>
        <p>This is some content inside a card. Cards can hold different
content and have borders with padding.</p>
        <button class="button">Click Me</button>
    </div>

    <div class="card">
        <h2>Another Card</h2>
        <p>This card contains a warning message below.</p>
        <p class="warning">This is a warning message!</p>
    </div>

</body>
</html>

```

Output:



This approach is commonly used for grouping elements that should share the same style and helps with maintaining cleaner, more organized code.

Advantages of Class Selectors

- ✓ **Reusability:** You can apply the same class to multiple elements, making your CSS more efficient and reducing redundancy.
- ✓ **Specificity:** Classes offer a higher level of targeting than tag selectors, but are still more flexible than IDs, which are meant to be unique for a single element.
- ✓ **Flexibility:** You can easily change the style of all elements with a particular class by modifying the CSS rule for that class.

ID Selector (#)

In CSS, an ID selector is used to target a specific element on an HTML document using its id attribute. The ID selector is defined with a hash symbol (#) followed by the id value. Each id in an HTML document should be unique, meaning it should only be assigned to one element.

Since id selector targets specific element, its used if we want to give some unique look and feel to some specific element.

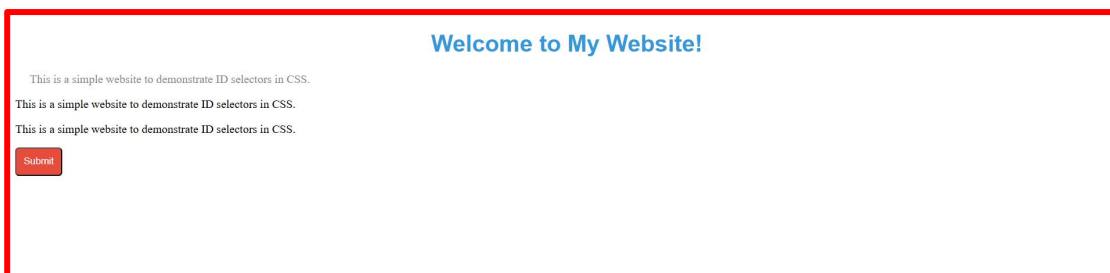
Process of using ID selector.

1. In HTML, you apply a class to an element using the ID attribute. An element can have single ID and ID should be unique across the website.
2. In CSS, you use a hash (#) followed by the id value to create a id selector.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title> ID Selector Example</title>
  <style>
    #title {
      color: #3498db; /* Blue text color */
      font-family: 'Arial', sans-serif;
      text-align: center;
    }
    #intro {
      font-size: 16px;
      color: #7f8c8d; /* Grayish text color */
      text-align: justify;
      margin: 0 20px;
    }
    #submit-btn {
      background-color: #e74c3c; /* Red background */
      color: white;
      padding: 10px;
      border-radius: 5px;
    }
  </style>
</head>
<body>
  <h1 id="title">Welcome to My Website!</h1>
  <p id="intro">This is a simple website to demonstrate ID selectors in
CSS.</p>
  <p>This is a simple website to demonstrate ID selectors in CSS.</p>
  <p>This is a simple website to demonstrate ID selectors in CSS.</p>
  <button id="submit-btn">Submit</button>
</body>
</html>
```

Output:



Advantages of ID Selector

1. **Uniqueness and Specificity:**

- ✓ ID selectors are highly specific because the id attribute should be unique within the document. This ensures that the style applies to a single, distinct element on the page.
- ✓ Their specificity gives them priority over other types of selectors like class selectors or element selectors.

2. **Fast and Efficient:**

- ✓ Browsers can quickly locate elements with an ID due to their uniqueness, leading to faster rendering compared to more general selectors.

3. **Easy to Target:**

- ✓ ID selectors make it easy to directly target and style a specific element without affecting others. This is useful when you want to apply unique styles to an individual element.

4. **Clear Intent:**

- ✓ Using an ID selector makes it clear that you're styling a particular element with a unique purpose, which can improve the readability and organization of your code.

Disadvantages of ID Selector

1. **Limited Reusability:**

- ✓ An ID selector can only be used once per document because id values must be unique. If you want to apply the same styles to multiple elements, you cannot reuse an ID selector (you would need to use a class selector instead).

2. **Overuse Can Lead to Specificity Issues:**

- ✓ Since ID selectors have high specificity, they can override other selectors, even those in classes or tag selectors. This can lead to difficulty in debugging or overriding styles when necessary, especially in larger projects with complex stylesheets.

3. **Not Ideal for Global Styles:**

- ✓ Since each ID is unique, you cannot use ID selectors for general, reusable styles across multiple elements. This makes them less useful for creating global or shared styles across a website. Class selectors are typically better for that purpose.

4. **Potential for Confusion:**

- ✓ If you have multiple IDs that look similar or aren't named clearly, it could confuse developers working on the project, making it harder to maintain the code in the long term.

5. **Overuse Can Lead to Maintenance Challenges:**

- ✓ If your project grows and more unique IDs are introduced, you might find it difficult to manage, especially if IDs are applied dynamically or programmatically. This can cause style conflicts and maintenance difficulties.

Attribute Selector

An attribute selector targets HTML elements based on the presence or value of an attribute. This allows you to apply styles to elements without having to rely on classes or IDs.

Descendant Selector (space)

The descendant selector selects elements that are inside (nested within) a specific element. It applies styles to any matching elements, regardless of how deep they are nested inside the parent.

Child Selector (>)

The child selector selects elements that are direct children of another element. This is more specific than the descendant selector. It only selects the immediate children, not deeper nested elements.

Pseudo Classes (:)

A pseudo class selector selects elements based on their state or a special condition, rather than their tag name or attributes. A pseudo-class is used to style elements in a specific state, such as when a user interacts with them (hover, focus, etc.) or to select elements based on their position in the document. E.g.

:hover – Applied when the user hovers over an element.

:focus – Applied when an element (like an input) is focused.

:first-child – Targets the first child of a parent element. Etc.

Combinators

Combinators combine different selectors to create more specific and targeted selections.

There are generally four types of combinator selectors.

1. **Descendant Selector (space):** (already described above)
2. **Child Selector (>):** (already described above)
3. **Adjacent Sibling Selector (+):** Selects the element that is immediately after another sibling element (e.g., h1 + p selects the <p> that comes immediately after an <h1>).
4. **General Sibling Selector (~):** Selects an element that is a sibling of another element, but it doesn't have to be immediately adjacent (e.g., h1 ~ p selects all <p> elements that are siblings of an <h1>, even if there are other elements in between).

Example:

Style.css

```
/* 1. Attribute Selector: Target anchor link with specific href */
a[href="https://example.com"] {
  color: red; /* Change the color of the link */
}

/* 2. Descendant Selector (space): Target p inside div (nested p) */
div p {
  color: blue; /* Change text color of all paragraphs inside divs */
}
```

```

}

/* 3. Child Selector (>): Target p that is a direct child of div */
div > p {
    font-size: 25px; /* Apply font size to p directly inside div */
}

/* 4. Pseudo-Classes: */
a:hover {
    text-decoration: underline; /* Underline links when hovered */
}

p:first-child {
    font-weight: bold; /* Make the first paragraph inside each parent element bold */
}

/* 5. Combinators: */

/* Adjacent Sibling Selector (+): Apply styles to the paragraph immediately after h1 */
h1 + p {
    margin-top: 0; /* Remove top margin for p after h1 */
}

/* General Sibling Selector (~): Apply styles to all p elements after h1 */
h1 ~ p {
    color: green; /* Change color of all p elements that follow h1 */
}

```

Index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CSS Selectors Example</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="container">
        <h1>Heading</h1>
        <p>This is a paragraph inside a div.</p>
        <a href="https://example.com" class="link">Click here to visit
Example</a>

        <div class="nested">
            <p>This is a paragraph inside a nested div.</p>
        </div>
    </div>

</body>
</html>

```

Output:

Heading

This is a paragraph inside a div.

[Click here to visit Example](#)

This is a paragraph inside a nested div.

CSS Colors and CSS Color models

CSS uses colors extensively to style websites and impact user experience. There are multiple ways to specify these colors, each using a different color model. These models provide various methods, from simple color names to more complex systems involving numbers and percentages, for defining the precise shade you want.

These color models are applied to different elements for text, background, border color etc.

The most commonly used color models are as follows.

1. **Named Colors:** This is the simplest way to define colors in CSS. CSS supports a set of predefined color names (like red, blue, green, black, etc.). There are 140 named colors available in CSS that you can use directly in your styles.

Example:

```
p { color: red; }
```

The above css will apply red color to the text of paragraph.

2. **Hexadecimal Color (Hex):** A hexadecimal color code consists of six characters preceded by a #. The first two characters represent the red component, the next two represent green, and the last two represent blue (RGB). The range for each color component is from 00 to FF (in hexadecimal), which corresponds to a value between 0 and 255 in decimal. Since its color number is not limited (around 16 million colors), it is one of the most widely used color models in css.

Format: #RRGGBB

Example:

```
h1 { color: #FF8C00; /* A shade of red-orange */ }
```

The above css will apply color with hex value FF8C00 for the text on h1 element

3. **RGB (Red, Green, Blue):** The RGB model defines colors using the intensity of red, green, and blue components. Each component is represented by a decimal value ranging from 0 to 255.

Format: rgb(red-component, green-component, blue-component)

Example:

```
p { background-color: rgb(255, 165, 0); }
```

The above css will apply background color with a shade of orange with rgb value with red component 255, green component 165 and blue component 0.

4. **RGBA (Red, Green, Blue, Alpha):** RGBA is an extension of the RGB model that includes an additional parameter for opacity or transparency called Alpha. The alpha value is a number between 0 (completely transparent) and 1 (completely opaque). This model is mainly used for overlays or subtle backgrounds.

Format: rgba(red-component, green-component, blue-component, alpha)

Example:

```
p { background-color: rgba(255, 0, 0, 0.5); }
```

The above css will apply red background color (no blue and green component) with 50% opacity to paragraph elements.

5. **HSL (Hue, Saturation, Lightness):** The HSL model defines colors based on hue, saturation, and lightness.
 - ✓ **Hue:** The color's position on the color wheel (0-360 degrees).
 - ✓ **Saturation:** The intensity of the color (0-100%).
 - ✓ **Lightness:** The brightness of the color (0-100%).

Format: hsl(hue, saturation%, lightness%)

Example:

```
div { background-color: hsl(120, 100%, 50%); }
```

The above css will apply background color with 120 degree hue in color wheel with 100% saturation and 50% lightness.

6. **HSLA (Hue, Saturation, Lightness, Alpha):** HSLA is an extension of the HSL model that includes an additional parameter for opacity or transparency called Alpha. The alpha value is a number between 0 (completely transparent) and 1 (completely opaque). This model is mainly used for overlays or subtle backgrounds.

Format: `hsla(hue, saturation%, lightness%, alpha)`

Example:

```
div { background-color: hsl(120, 100%, 50%, 0.5);
```

The above css will apply background color with 14 degree hue in color wheel with 100% saturation and 50% lightness with 50% opacity.

Among above 6 color models, you choose any model according to your requirement. Below are some basic use cases where each color models are used.

- ✓ **Named colors:** Use for quick styling and basic color choices.
- ✓ **Hex codes:** Use for a wide range of colors and precise control.
- ✓ **RGB:** Use for intuitive color definition and precise control.
- ✓ **RGBA:** Use when you need semi-transparent colors.
- ✓ **HSL:** Use when you want to adjust color properties like brightness and intensity easily.
- ✓ **HSLA:** Use when you need semi-transparent colors based on hue, saturation, and lightness.

CSS Backgrounds

In CSS, the background property is used to set the background effects for an element. This property can take multiple values to control the background color, image, position, repeat behavior, size, and more.

There are multiple css background properties, they are listed below

- ✓ **background-color :** Sets the background color of an element. It's a simple and fundamental way to style the background. This can be any valid CSS color (hex, RGB, rgba, etc.). The color will be displayed if no image is set.
- ✓ **background-image :** Specifies the image to use as the background. You can use either a URL pointing to an image file or a gradient function like linear-gradient or radial-gradient.
- ✓ **background-position :** This property defines the initial position of the background image. It can accept values like pixel values, percentages, or keywords (like top, right, bottom, left, and center). You can specify both horizontal and vertical positioning.
- ✓ **background-size:** This property defines the size of the background image. You can use specific measurements like pixels or percentages, or use the

- keywords contain (scales the image to fit the container) and cover (scales the image to cover the entire container without distortion, but may crop it).
- ✓ background-repeat: · This property controls how the background image is repeated. This property controls whether the background image repeats itself horizontally or vertically, or both in case when single image can not fill the space. CSS will repeat the image to fill the space based on the value of background-repeat. If the image is large enough to cover the area, it will only appear once. It can have the following values:
 - ◆ repeat: The background image is repeated both horizontally and vertically (default behavior).
 - ◆ no-repeat: The background image is not repeated.
 - ◆ repeat-x: The background image is repeated horizontally only.
 - ◆ repeat-y: The background image is repeated vertically only.
 - ✓ background-attachment: This property controls whether the background image scrolls along with the content or remains fixed during scrolling. It can have the following values:
 - ◆ scroll: The background image scrolls along with the element's content (default behavior).
 - ◆ fixed: The background image stays fixed relative to the viewport when the content is scrolled.
 - ◆ local: The background image scrolls with the element's content (within the element itself).
 - ✓ background-origin : This property sets the initial position of the background image. It can accept values like pixel values, percentages, or keywords (like top, right, bottom, left, center or combinations like top right, bottom left etc.). You can specify both horizontal and vertical positioning.
 - ✓ background-clip : This property controls whether the background image scrolls along with the content or remains fixed during scrolling. It can have the following values:
 - ◆ scroll: The background image scrolls along with the element's content (default behavior).
 - ◆ fixed: The background image stays fixed relative to the viewport when the content is scrolled.
 - ◆ local: The background image scrolls with the element's content (within the element itself).
 - ✓ background (shorthand) : You can combine all these properties into a single background declaration: This shorthand is often preferred for conciseness. Remember to specify the values in the correct order. The order is
 - ◆ background-color:
 - ◆ background-image:
 - ◆ background-repeat:
 - ◆ background-attachment:
 - ◆ background-position:
 - ◆ background-size: This is often placed separately after background property.

It's important to note that you don't have to include all of these values. You can omit any that you want to keep at their default values.

Background-color:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Selectors Example</title>
  <style>
    h1 {
      background-color: yellow;
    }
    div {
      background-color: #115500;
      opacity: .7;
    }
    p {
      background-color: rgb(200, 150, 255);
    }
  </style>
</head>
<body>

  <div class="container">
    <h1>Heading</h1>
    <p>This is a paragraph inside a div.</p>
    <a href="https://example.com" class="link">Click here to visit
Example</a>
    <p>This is a paragraph inside a nested div.</p>
  </div>

</body>
</html>
```

Output:



Common Example :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Background Properties Example</title>
  <style>
    body {
      background-color: #f0f0f0; /* Light gray background for the
entire body */
      margin: 0; /* Remove default body margin */
      padding: 0; /* Remove default body padding */
    }

    .container {
      width: 80%; /* Container width */
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Heading</h1>
    <p>This is a paragraph inside a div.</p>
    <a href="https://example.com" class="link">Click here to visit
Example</a>
    <p>This is a paragraph inside a nested div.</p>
  </div>
</body>
</html>
```

```

        margin: 20px auto; /* Center the container */
        border: 1px solid #ccc; /* Container border */
        padding: 20px; /* Container padding */
    }

    .example {
        margin-bottom: 20px; /* Space between examples */
        padding: 10px; /* Padding within each example box */
        border: 1px solid #ddd; /* Border for each example box */
    }

    .example h2 {
        margin-top: 0; /* Remove default h2 margin */
    }

    #bg-color {
        background-color: lightblue; /* Sets the background color to
light blue */
    }

    #bg-image {
        background-image: url("onefiftybyonefifty.png"); /* Sets the
background image URL */
        background-repeat: no-repeat; /* Prevents the image from
repeating */
        background-position: center; /* Centers the background image */
        background-size: contain; /* Makes sure the entire image is
visible, may add blank space */
    }

    #bg-repeat {
        background-image: url("fiftybyfifty.png"); /* Sets the
background image URL */
        background-repeat: repeat-x; /* Repeats the image only
horizontally */
    }

    #bg-position {
        background-image: url("fiftybyfifty.png"); /* Sets the
background image URL */
        background-repeat: no-repeat; /* Prevents the image from
repeating */
        background-position: right bottom; /* Positions the image at
the bottom right */
    }

    #bg-size-cover {
        background-image: url("onefiftybyonefifty.png"); /* Sets the
background image URL */
        background-size: cover; /* Makes the image cover the entire
container, may crop parts of the image */
        background-repeat: no-repeat; /* Prevents the image from
repeating */
        height: 200px; /* Sets a height to demonstrate the effect */
    }

    #bg-size-contain {
        background-image: url("onefiftybyonefifty.png"); /* Sets the
background image URL */

```

```

        background-size: contain; /* Makes sure the entire image is
visible within the container */
        background-repeat: no-repeat; /* Prevents the image from
repeating */
        height: 200px; /* Sets a height to demonstrate the effect */
    }

    #bg-attachment {
        /*background-image: url("https://dummyimage.com/50"); /* Sets
the background image URL */
        background-image: url("fiftybyfifty.png"); /* Sets the
background image URL */
        background-attachment: fixed; /* Fixes the background image so
it doesn't scroll */
        height: 500px; /* Sets a height to demonstrate the effect */
    }

    #bg-shorthand {
        background: #ffddaa url("fiftybyfifty.png") no-repeat center
center fixed; /* Shorthand for setting multiple background properties */
        background-size: cover; /* Often applied separately */
        height: 200px; /* Sets a height to demonstrate the effect */
    }
</style>
</head>
<body>
    <div class="container">
        <h1>CSS Background Properties Example</h1>

        <div class="example" id="bg-color">
            <h2>background-color</h2>
            <p>This div has a light blue background.</p>
        </div>

        <div class="example" id="bg-image">
            <h2>background-image</h2>
            <p>This div has a background image, not repeated, centered, and
contained.</p>
        </div>

        <div class="example" id="bg-repeat">
            <h2>background-repeat</h2>
            <p>This div's background image repeats only horizontally.</p>
        </div>

        <div class="example" id="bg-position">
            <h2>background-position</h2>
            <p>This div's background image is positioned at the bottom
right.</p>
        </div>

        <div class="example" id="bg-size-cover">
            <h2>background-size: cover</h2>
            <p>This div's background image covers the entire area.</p>
        </div>

        <div class="example" id="bg-size-contain">
            <h2>background-size: contain</h2>

```

```

        <p>This div's background image is contained within the
area.</p>
    </div>

    <div class="example" id="bg-attachment">
        <h2>background-attachment: fixed</h2>
        <p>This div's background image is fixed and does not scroll
with the content.</p>
    </div>

    <div class="example" id="bg-shorthand">
        <h2>Background Shorthand</h2>
        <p>This div uses the background shorthand property.</p>
    </div>
</div>
</body>
</html>

```

CSS Borders

borders are used to create a visible line around elements. The border property is shorthand for setting multiple border-related properties like border-width, border-style, and border-color. You can apply borders to all HTML elements (such as div, p, button, etc.) to visually separate content.

The CSS border properties are used to specify the style, color and size of the border of an element. The CSS border properties are given below

- border-width
- border-color
- border-style
- border-radius

border-width

border-width defines the thickness of the border. You can set different values for each side (top, right, bottom, left). The border-width can be set using keywords like thin, medium, and thick, or specific pixel values.

border-color

Defines the color of the border. You can use any color model (named color, RGB, RGBA, HSL, HSLA) to represent the color. **The border-color property is not used alone. It is always used with other border properties like "border-style" property to set the border first otherwise it will not work.**

You can set color for all the sides of the border or at any individual side as per requirement.

Example:

```

border-color: red; /* All sides have a red border */
border-top-color: green; /* Top border is green */
border-right-color: blue; /* Right border is blue */
border-bottom-color: yellow; /* Bottom border is yellow */
border-left-color: purple; /* Left border is purple */

```

CSS border-style

The Border style property is used to specify the border type which you want to display on the web page. There are some border style values which are used with border-style property to define a border.

Value	Description
none	Default value. Specifies no border
hidden	The same as "none", except in border conflict resolution for table elements
dotted	Specifies a dotted border
dashed	Specifies a dashed border
solid	Specifies a solid border
double	Specifies a double border
groove	Specifies a 3D grooved border. The effect depends on the border-color value
ridge	Specifies a 3D ridged border. The effect depends on the border-color value
inset	Specifies a 3D inset border. The effect depends on the border-color value
outset	Specifies a 3D outset border. The effect depends on the border-color value
initial	Sets this property to its default value.
inherit	Inherits this property from its parent element.

Border-radius

The border-radius property is used to create rounded corners on any element. It works on all four corners of an element and can be set individually. radius can be a length (like px, em, etc.) or a percentage.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Border Styles</title>
  <style>
    .box {
      width: 150px;
      height: 50px;
      margin: 10px;
      border: 3px; /* Initial border width */
      border-color: #FF3330; /* Initial border color */
    }

    .solid {
      border-style: solid;
      border-color: green;
      border-radius: 10px;
    }
  </style>
</head>
```

```

        .dotted { border-style: dotted; border-color: rgb(1, 200, 255);}
        .dashed { border-style: dashed; }
        .double { border-style: double; border-top-color: greenyellow;}
        .groove { border-style: groove; }
        .ridge { border-style: ridge; }
        .inset { border-style: inset; }
        .outset { border-style: outset;}

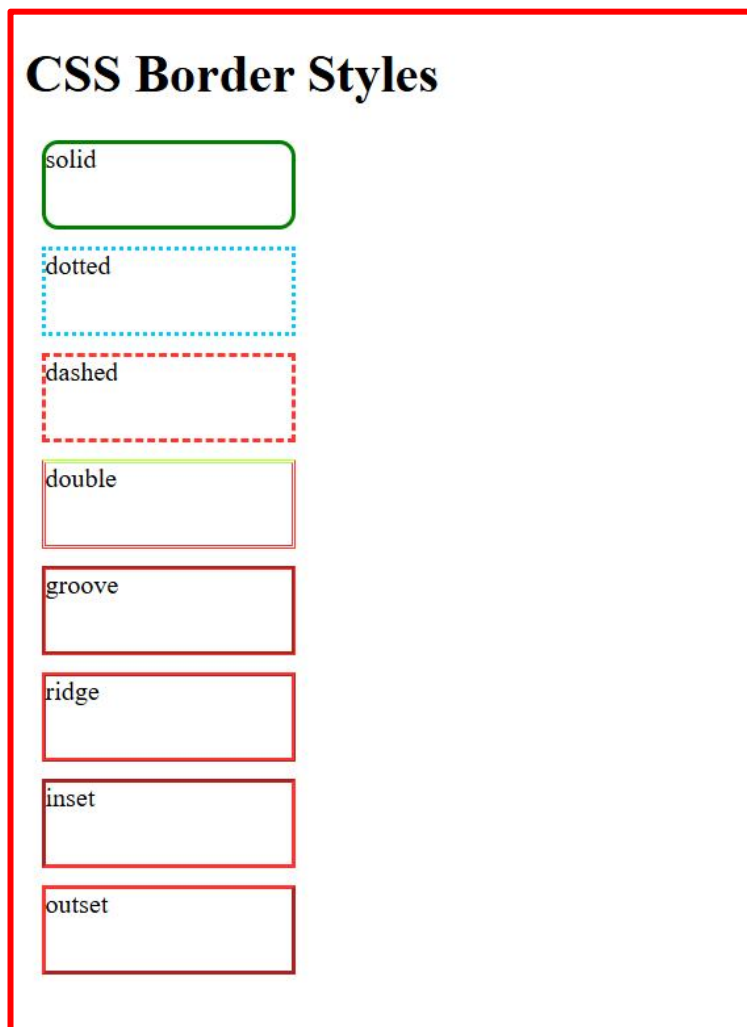
    </style>
</head>
<body>
    <h1>CSS Border Styles</h1>

    <div class="box solid">solid</div>
    <div class="box dotted">dotted</div>
    <div class="box dashed">dashed</div>
    <div class="box double">double</div>
    <div class="box groove">groove</div>
    <div class="box ridge">ridge</div>
    <div class="box inset">inset</div>
    <div class="box outset">outset</div>

</body>
</html>

```

Output:



Border Shorthand

The border shorthand in CSS is a convenient way to set all the border properties of an element in a single line. This shorthand combines three main border properties:

- border-width: Specifies the thickness of the border.
- border-style: Defines the style of the border (solid, dotted, dashed, etc.).
- border-color: Specifies the color of the border.

These properties should be in correct order in order to work.

Example :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Border Style</title>
  <style>
    div {
      border: 2px dotted #3498db;
    }
  </style>
</head>
<body>
  <h1>CSS Border shorthand</h1>

  <div>shorthand</div>

</body>
</html>
```

Output



Text manipulation in CSS

In CSS, text manipulation means influencing the text's appearance on a web page, involving styles such as fonts, colors, sizes, alignments, spacing, text transformation, or other visual effects. CSS has different properties to control, read, or beautify text in webpages.

Some of the most common text manipulation properties in CSS are described below.

Text Color

The color property changes the color of the text. You can specify colors using any of the color models (named color, rgb, rgba, hex color code, hsl, hsla)

Eg.

```
p {  
  color: #333333;  
}
```

Text Alignment/ Text Direction

The text-align property defines the horizontal alignment of text inside a block element. It can be set to

- left,
- right,
- center, or
- justify.

Eg.

```
h1 {  
  text-align: center;  
}
```

Line Height

The line-height property controls the vertical spacing between lines of text. It's especially useful for improving the readability of paragraphs.

Eg.

```
p {  
  line-height: 1.6;  
}
```

Text Transform

The text-transform property allows you to control the capitalization of text. Common values include uppercase, lowercase, and capitalize.

Eg.

```
h1 {  
  text-transform: uppercase;  
}
```

Letter Spacing

The letter-spacing property controls the space between characters. It's often used for stylizing text in headlines or logos.

Eg.

```
h1 {  
  letter-spacing: 2px;  
}
```

Text Decoration

The text-decoration property is used to add decorative lines to text. It can take values like underline, overline, line-through, or none.

Eg.

```
p {  
  text-decoration: underline;  
}
```

Text Intendation

The text-indent property sets the amount of space to indent the first line of text in a block.

Eg.

```
p {
  text-indent: 30px;
}
```

Text Shadow

The `text-shadow` property adds a shadow effect to text. You can define the shadow's horizontal and vertical offsets, blur radius, and color.

Eg.

```
h2 {
  text-shadow: 2px 2px 5px rgba(0, 0, 0, 0.5);
}
```

White Space

The `white-space` property defines how whitespace (spaces, tabs, newlines) is handled inside an element. It can take values like `normal`, `nowrap`, `pre`, and others.

Eg.

```
pre {
  white-space: pre-wrap;
}
```

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Text Manipulation Example</title>
    <style>
        /* General Body Styling */
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f4;
            color: #333;
            margin: 0;
            padding: 20px;
        }

        /* Heading Styling */
        h1 {
            text-align: center;
            font-size: 36px;
            color: #2c3e50;
            text-transform: uppercase;
            letter-spacing: 5px;
        }

        /* Paragraph Styling */
        p {
            line-height: 1.6;
            color: #555;
            max-width: 800px;
```

```

        margin: 20px auto;
        white-space: pre-wrap;
    }

    /* Emphasized (Italicized) Text */
    em {
        color: #e74c3c; /* Red color for emphasis */
    }

    /* Underlined Link Styling */
    a {
        color: #2980b9;
        text-decoration: underline;
    }

    /* Adding Text Shadow */
    .shadow-text {
        text-shadow: 2px 2px 5px rgba(0, 0, 0, 0.3);
    }

    /* Adjusting Text Alignment */
    .center-text {
        text-align: center;
    }

    /* Text Indentation */
    .indent-text {
        text-indent: 30px;
    }
</style>
</head>
<body>

    <h1>Text Manipulation in CSS</h1>

    <p class="center-text">Welcome to this tutorial on text manipulation
    using HTML and CSS. Here we will explore how to change the appearance of
    text, such as making it italic, bold, underlined, and adding text
    shadows.</p>

    <p class="indent-text">
        Lorem ipsum dolor sit amet, <em>consectetur adipiscing elit</em>,
        sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim
        ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip
        ex ea commodo consequat. <span class="bold-text">Duis aute irure
        dolor</span> in reprehenderit in voluptate velit esse cillum dolore eu
        fugiat nulla pariatur.
    </p>

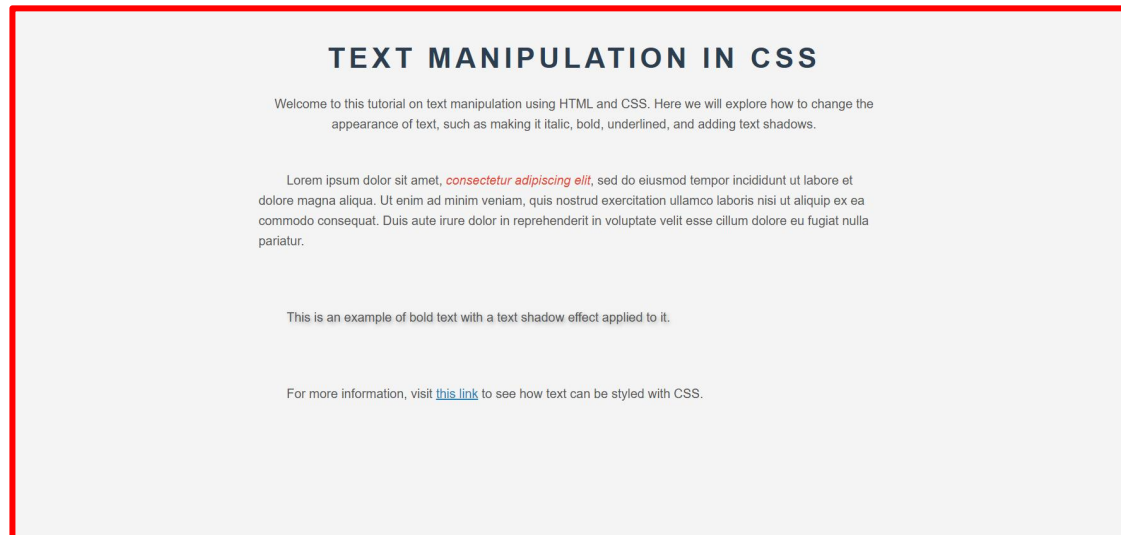
    <p class="shadow-text">
        This is an example of <span class="bold-text">bold text</span> with
        a <span class="shadow-text">text shadow</span> effect applied to it.
    </p>

    <p>
        For more information, visit <a href="https://www.example.com">this
        link</a> to see how text can be styled with CSS.
    </p>

```

```
</body>
</html>
```

Output:



Font Handling in CSS

CSS fonts control how text appears on a webpage. With CSS, you can specify various properties like font family, size, weight, style, and line height to create visually appealing and readable typography. Choosing the right font and style is very crucial for the readability of text on a page.

CSS provide several properties for styling the font of the text, including changing their face, controlling their size and boldness, managing variant, and so on.

The font properties are:

- font-family,
- font-style,
- font-weight,
- font-size, and
- font (Shorthand)

Font-family

The font-family property defines which font(s) should be used for text. You can provide a list of fonts as a fallback mechanism in case the preferred font is unavailable.

```
body {
  font-family: "Arial", sans-serif;
}
```

Font-style

The font-style property defines the style of the font, typically italic or normal.

```
em {
  font-style: italic; /* Italicized text */
}

p {
  font-style: normal; /* Normal text */
}
```

Font-weight

The font-weight property controls the thickness of the text. It can accept values like normal, bold, or numeric values (100 to 900).

```
p {
  font-weight: bold; /* Bold text */
}

strong {
  font-weight: 700; /* Equivalent to bold */
}
```

Font-size

The font-size property controls the size of the text. You can set the size in several units, including pixels (px), ems (em), and percentages (%).

```
h1 {
  font-size: 32px; /* Pixels */
}

p {
  font-size: 1.2em; /* Relative to the parent element */
}
```

Font (shorthand)

Combines multiple font properties into one

```
p { font: italic bold 16px/1.5 "Arial", sans-serif; } /* style weight
size/line-height family */
```

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Text Manipulation Example</title>
  <style>
    /* Styling for the whole page */
    body {
      background-color: #eaf6f6;
      padding: 20px;
      margin: 0;
      color: #333;
    }

    /* Styling for the paragraph */
```

```

    p {
        font-family: Arial, sans-serif;
        font-size: 18px;
        line-height: 1.6;
        color: #555;
        max-width: 800px;
        margin: 20px auto;
    }

    /* Styling for the emphasized (italicized) text */
    em {
        font-style: italic; /* Italicized text */
        color: #e74c3c; /* Red color for emphasis */
    }

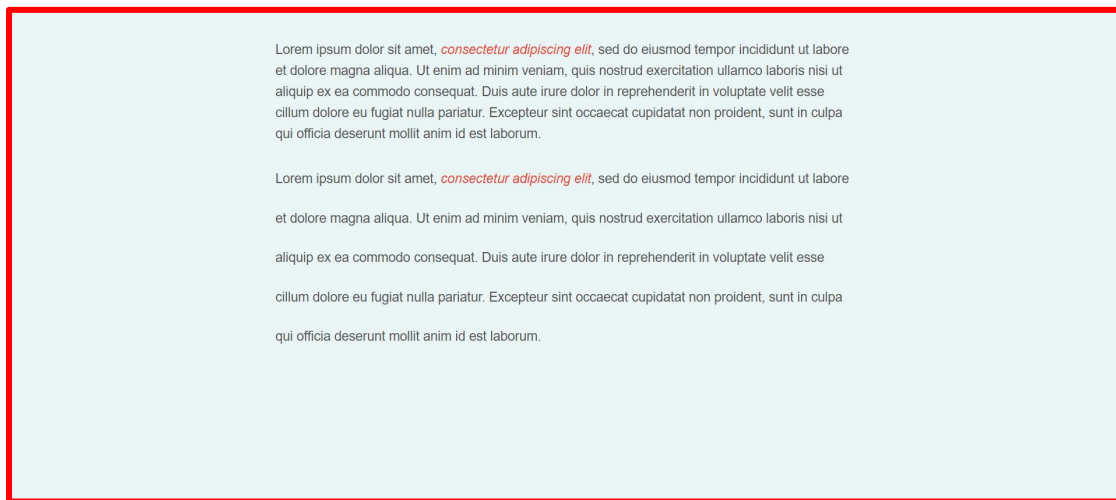
    div p {
        font-family: sans-serif, Arial;
        line-height: 3em;
        font-weight: strong;
    }
</style>
</head>
<body>

    <p>
        Lorem ipsum dolor sit amet, <em>consectetur adipiscing elit</em>,
        sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
        Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris
        nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in
        reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla
        pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui
        officia deserunt mollit anim id est laborum.
    </p>
    <div>
        <p>
            Lorem ipsum dolor sit amet, <em>consectetur adipiscing
            elit</em>, sed do eiusmod tempor incididunt ut labore et dolore magna
            aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris
            nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in
            reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla
            pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui
            officia deserunt mollit anim id est laborum.
        </p>
    </div>

</body>
</html>

```

Output:



List CSS

There are three common properties specific to styling lists: list-style-type, list-style-position, and list-style-image. There is also a shorthand property which includes all three. The properties and their possible values are listed below.

Property	Description	Values for ul	Values for ol
list-style-type	Defines the type of list item marker.	disc, circle, square, none	decimal (default) decimal-leading-zero lower-roman upper-roman lower-greek lower-latin upper-latin armenian georgian lower-alpha upper-alpha none
list-style-position	list-style-position controls whether the list marker appears inside or outside of each list item element ().	outside, inside	outside, inside
list-style-image	Specifies an image to be used as a list marker.	url(<image>), none	url(<image>), none
list-style	Shorthand for setting the list marker type, position, and image.		

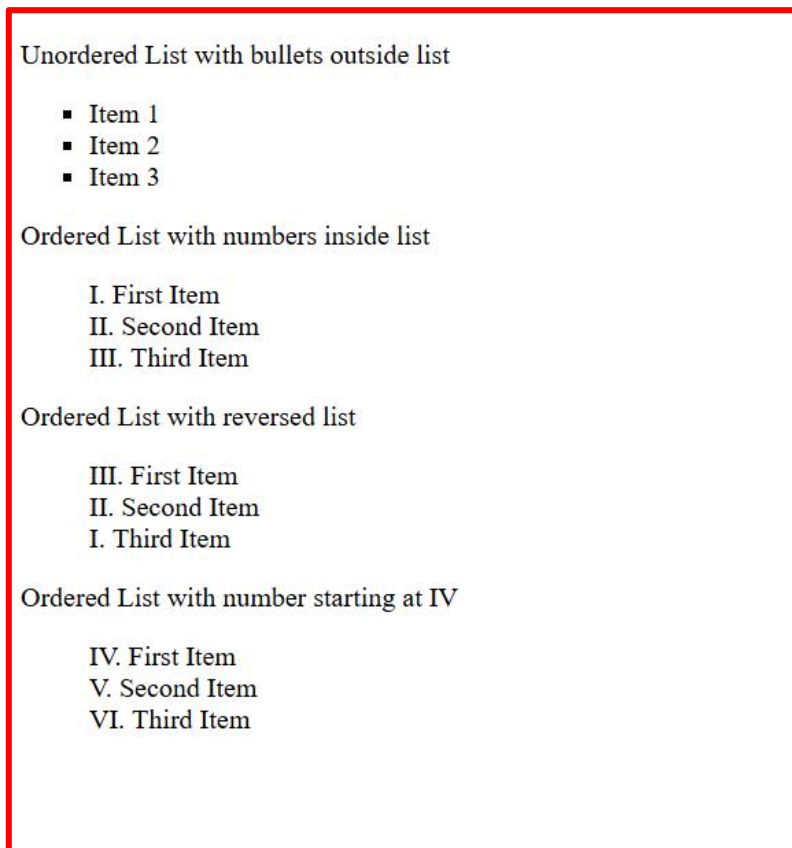
Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>List CSS Example</title>
  <style>
    ul {
      list-style-type: square;
      list-style-position: outside;
    }

    ol {
      list-style-type: upper-roman;
      list-style-position: inside;
    }
  </style>
</head>
<body>
  <p>Unordered List with bullets outside list</p>
  <ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
  </ul>
  <p>Ordered List with numbers inside list</p>
  <ol>
    <li>First Item</li>
    <li>Second Item</li>
    <li>Third Item</li>
  </ol>
  <p>Ordered List with reversed list</p>
  <ol reversed>
    <li>First Item</li>
    <li>Second Item</li>
    <li>Third Item</li>
  </ol>
  <p>Ordered List with number starting at IV</p>
  <ol start=4>
    <li>First Item</li>
    <li>Second Item</li>
    <li>Third Item</li>
  </ol>

</body>
</html>
```

Output:



CSS Tables:

CSS tables are used to style HTML tables, making them look neat and organized for clear data presentation.

Styling the table

Property	Description	Values
border	Sets all border properties at once for the table.	<length>, none, solid, dotted, dashed, etc.
border-collapse	Whether table borders are collapsed (single border) or separated (double border).	collapse, separate
border-spacing	Space between borders of adjacent cells (only applies if border-collapse: separate).	<length> (e.g., 10px)
caption-side	Position of the <caption> relative to the table.	top, bottom
table-layout	Table layout algorithm. fixed is often used for predictable column widths.	auto, fixed

Styling <th> (Table Header Cells) and <td> (Table Data Cells):

Property	Description	Values
border	Sets all border properties at once for the cell.	(px,em) , none, solid, dotted, dashed, etc.
width	Width of the cell.	(px,em), percentage, auto
height	Height of the cell.	(px,em), percentage, auto
empty-cells	How to display empty cells (applies to both <th> and <td>).	show, hide
text-align	Horizontal text alignment within the cell.	left, right, center, justify, start, end
vertical-align	Vertical text alignment within the cell.	top, middle, bottom,(px,em), baseline
border-top	Border at the top of the cell.	(px,em), solid, dotted, none, etc.
border-right	Border at the right of the cell.	(px,em), solid, dotted, none, etc.
border-bottom	Border at the bottom of the cell.	(px,em), solid, dotted, none, etc.
border-left	Border at the left of the cell.	(px,em), solid, dotted, none, etc.
padding	Space inside the cell, between content and borders.	em, px, percentage, etc.

Styling <tr> (Table Header Cells) and <td> (Table Data Cells):

Property	Description	Values
height	Height of the row.	Px,em, percentage, auto
background-color	Background color of the row.	<color>, transparent
vertical-align	Vertical alignment of content within the row's cells.	top, middle, bottom, (px,em), baseline

Apart from the above properties, you can always add text and background effects for table, tr, th, td, caption etc.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Multiple Tables Example</title>
  <style>
    th, td {
      padding: 10px;
      text-align: left;
      vertical-align: middle; /* Example vertical alignment */
      border: 1px solid #ccc; /* Cell borders */
    }

    th {
      font-weight: bold;
    }
  </style>
</head>
<body>
  <table>
    <tr>
      <th>Header 1</th>
      <th>Header 2</th>
      <th>Header 3</th>
    </tr>
    <tr>
      <td>Data 1</td>
      <td>Data 2</td>
      <td>Data 3</td>
    </tr>
  </table>
</body>
</html>
```

```

/* Table 1 Styles */
#table1 {
    table-layout: fixed; /* Fixed layout for predictable widths */
    border-collapse: separate; /* For border-spacing to work */
    border-left: dotted;
    border-right: dashed;
    border-color: red;
}
#table1 th {
    width: 200px; /* example fixed column width */
}
#table1 caption {
    caption-side: top; /* Caption at the top */
    font-style: italic;
    padding-bottom: 5px;
}
#table1 tr { /* Or target a specific row if needed */
    height: 100px; /* Or whatever height you need */
}
#table1 td {
    vertical-align: top;
    background-color: aquamarine;
}

/* Table 2 Styles */
#table2 {
    table-layout: auto; /* Auto layout */
    border-collapse: collapse; /* For border-spacing to work */
}
#table2 caption {
    caption-side: top; /* Caption at the bottom */
    text-align: right;
    padding-top: 5px;
}
#table2 td {
    text-align: center;
    vertical-align: baseline;
}

/* Table 3 Styles */
#table3 {
    empty-cells: hide; /* Hide empty cells */
    border-collapse: separate; /* For border-spacing to work */
    margin: 20px;
}
#table3 caption {
    caption-side: bottom; /* Caption at the bottom */
    text-align: right;
    padding-top: 5px;
}
</style>
</head>
<body>

    <table id="table1">
        <caption>Table with Fixed Layout</caption>
        <thead>
            <tr>

```

```

        <th>Column 1</th>
        <th>Column 2</th>
        <th>Column 3</th>
    </tr>
</thead>
<tbody>
    <tr>
        <td>Data 1</td>
        <td>Data 2</td>
        <td>Data 3</td>
    </tr>
    <tr>
        <td>Data 4</td>
        <td>Data 5</td>
        <td>Data 6</td>
    </tr>
</tbody>
</table>

```

```

<table id="table2">
    <caption>Table with Auto Layout</caption>
    <thead>
        <tr>
            <th>Name</th>
            <th>Age</th>
            <th>City</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Alice</td>
            <td>30</td>
            <td>New York</td>
        </tr>
        <tr>
            <td>Bob</td>
            <td>25</td>
            <td>London</td>
        </tr>
    </tbody>
</table>

```

```

<table id="table3">
    <caption>Table with Hidden Empty Cells</caption>
    <thead>
        <tr>
            <th>Product</th>
            <th>Price</th>
            <th>Quantity</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Laptop</td>
            <td>1200</td>
            <td></td> </tr>
        <tr>
            <td>Mouse</td>
            <td>25</td>

```

```

        <td>10</td>
      </tr>
    </tbody>
  </table>

</body>
</html>

```

Output:

Table with Fixed Layout		
Column 1	Column 2	Column 3
Data 1	Data 2	Data 3
Data 4	Data 5	Data 6

Table with Auto Layout		
Name	Age	City
Alice	30	New York
Bob	25	London

Product	Price	Quantity
Laptop	1200	
Mouse	25	10

Table with Hidden Empty Cells		

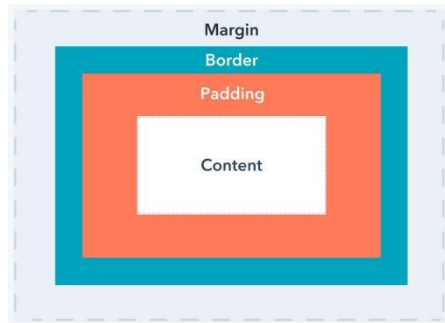
CSS Box Model

The CSS Box Model defines how elements are sized, positioned, and rendered on a webpage. When a browser loads an HTML document, it creates a DOM tree and assigns a box to each element. This box calculates the element's dimensions and position relative to its parent or the root <html> element, ensuring accurate layout and spacing.

The box model consists of several components:

- ✓ **Content:** This is the actual content of the element, such as text, images, etc. The width and height of the content area are set by the width and height properties in CSS.
- ✓ **Padding:** Padding is the space between the content and the border. It adds space inside the element, creating space around the content. Padding can be set for each side (top, right, bottom, left) using the padding property.

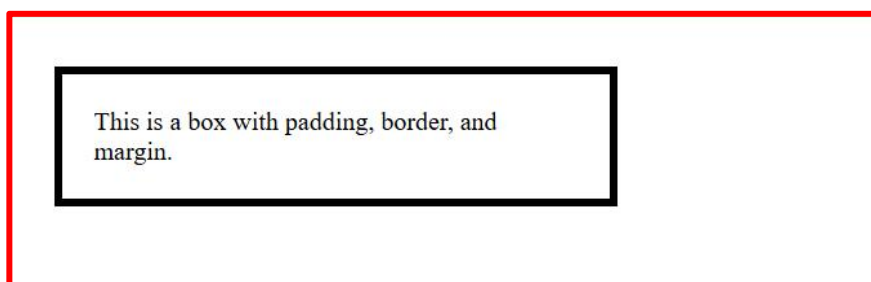
- ✓ **Border:** The border surrounds the padding (if any) and content. It can be styled with various widths, colors, and styles (solid, dotted, dashed, etc.) using the border property.
- ✓ **Margin:** Margin is the outermost space that separates the element from other elements. It creates space outside the element, and it can also be set for each side (top, right, bottom, left) using the margin property.



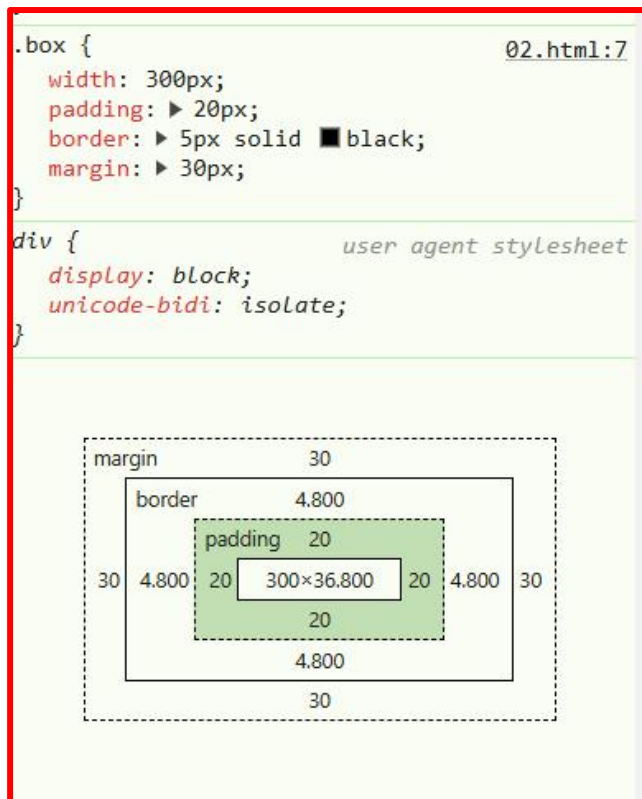
Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    .box {
      width: 300px;
      padding: 20px;
      border: 5px solid black;
      margin: 30px;
    }
  </style>
  <title>CSS Box Model Example</title>
</head>
<body>
  <div class="box">
    This is a box with padding, border, and margin.
  </div>
</body>
</html>
```

For the above example the output will be like this .



If we right open the developer options, (By right clicking and view page source or by right clicking and inspect, or by pressing f12 button). We can view the css and its box layout in developer tools window like the following by showing content area, margin, padding and border properties.



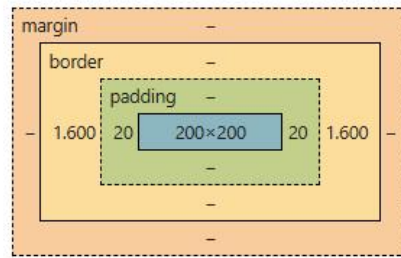
Box Sizing Property in CSS

There are two type's of box-sizing properties in CSS

1. Content-Box(default property): When the user set's the value of the box-sizing property for an element as `content-box` or even if user do not set's it ,it remains by default as `content-box` and in the actual height and width of the element the dimensions of the content area as well as the padding area is added to constitute the final dimensions of the element.

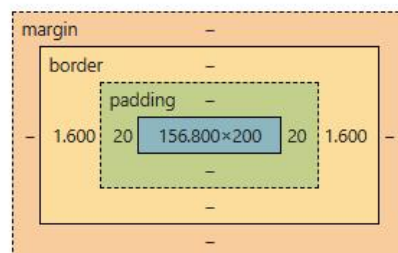
Example:

```
<style>
div {
  height: 200px;
  width: 200px;
  box-sizing: content-box;
  padding-left: 20px;
  padding-right: 20px;
  border-left: 2px solid red;
  border-right: 2px solid red;
}
</style>
</html>
```

2. Border-box: When the box-sizing property is set as border-box the actual dimensions of the element's remains same as that of the actual dimensions set by the user. The difference it makes is just that the size of the content area get's altered in a manner so that it could accommodate the padding area and the border area so the resultant could be equal to the actual dimensions entered by the user.

```
<style>
div {
  height: 200px;
  width: 200px;
  box-sizing: border-box;
  padding-left: 20px;
  padding-right: 20px;
  border-left: 2px solid red;
  border-right: 2px solid red;
}
</style>
```



Normal Flow Box Layout

The Normal Flow is the default behavior for elements in a document, where they are laid out in the order they appear in the HTML, starting from the top-left corner. Elements are stacked vertically (block-level elements) or arranged horizontally (inline elements).

- Block-level elements (like <div>, <p>, <h1>) occupy the full width available, creating a new line after them (a vertical stack).
- Inline elements (like , <a>) only take up as much width as needed and are placed next to each other in a horizontal line.

The basic box layout in normal flow would be a combination of padding, borders, and margins applied to elements.

CSS Positioning (Layout)

CSS positioning defines how elements are placed and positioned on a webpage. It controls how an element is placed in relation to other elements or its container. CSS positioning is a powerful tool that allows you to control the placement of elements on a web page. It goes beyond the normal flow of HTML, letting you precisely position elements relative to their parent, the document, or even the viewport.

There are five primary types of positioning in CSS:

1. Static (default positioning)
2. Relative
3. Absolute
4. Fixed
5. Sticky

Static (default positioning)

By default, the position property for all HTML elements in CSS is set to static. This means that if you don't specify any other position value or if the position property is not declared explicitly, it'll be static.

Visually, all elements follow the order of the HTML code, and in that way the typical document flow is created.

Elements appear one after the other – directly below one another, according to the order of the HTML code.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>CSS Positioning</title>
    <style>
      body {
        margin: 100px;
      }

      .parent {
        width: 500px;
        border: 1px solid red;
        margin: auto;
        text-align: center;
      }

      .child {
        border-radius: 10%;
        width: 100px;
        height: 100px;
      }
    </style>
  </head>
  <body>
    <div class="parent">
      <div class="child"></div>
    </div>
  </body>
</html>
```

```

margin: 20px;
}

.one {
background-color: blue;
}

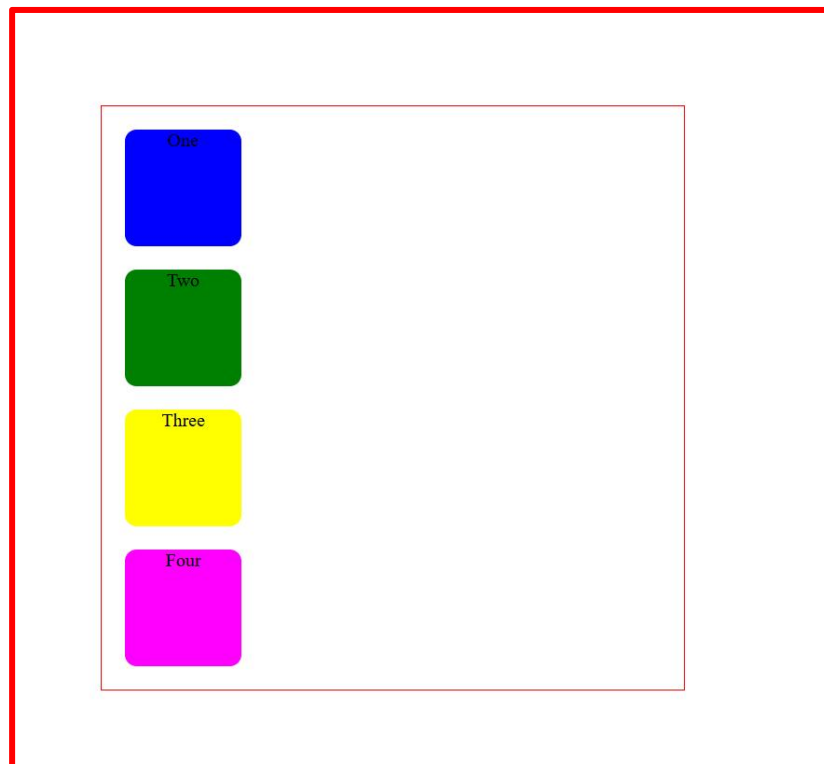
.two {
background-color: green;
}

.three {
background-color: yellow;
}

.four {
background-color: magenta;
}
</style>
</head>
<body>
  <div class="parent">
    <div class="child one">One</div>
    <div class="child two">Two</div>
    <div class="child three">Three</div>
    <div class="child four">Four</div>
  </div>
</body>
</html>

```

Output:



Relative

Relative Positioning is a CSS technique that changes the position of the element relative to the parent element and relative to itself and where it would usually be in the regular document flow of the page. This means that it's relative to its original position within the parent element.

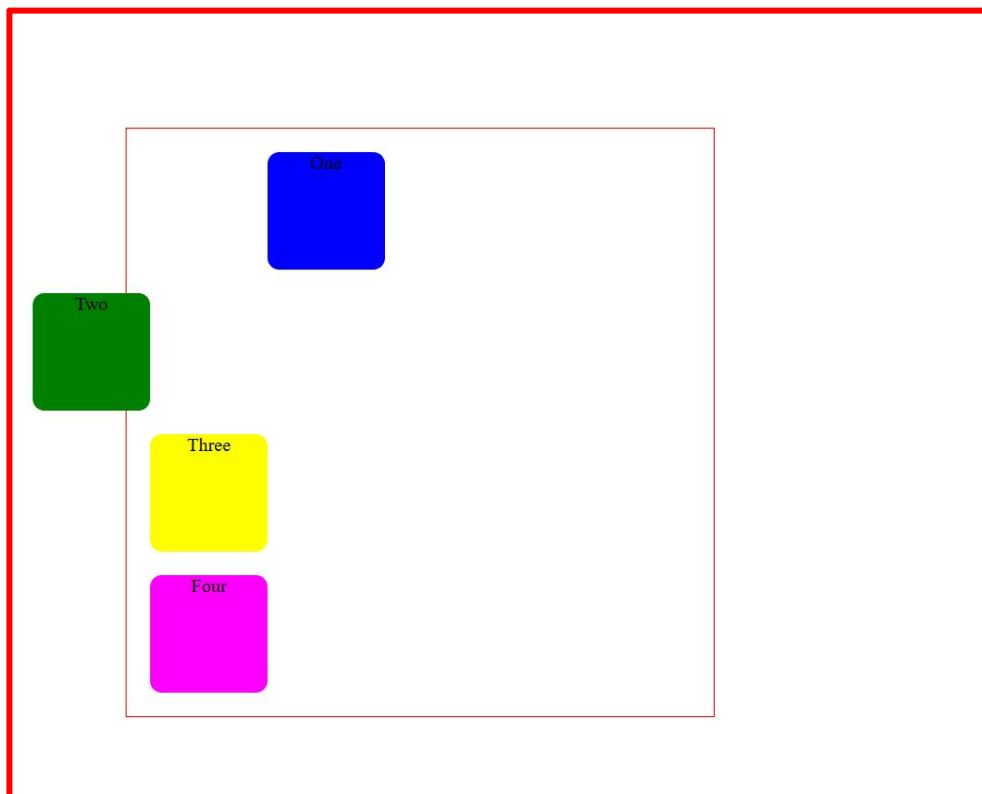
The syntax for relative positioning is `position: relative;` When you set the top, right, bottom, and left properties of an element with relative positioning, it moves from its original location. However, it's important to note that other elements on the page will not fill the gap left by the moved element.

Example:

In the above example, suppose we want to move box1 to right and box to left of original position, we modify the css as follows.

```
.one {  
  position: relative;  
  left: 100px;  
  background-color: blue;  
}  
  
.two {  
  position: relative;  
  right: 100px;  
  background-color: green;  
}
```

The output will be like follows.



Absolute

Absolute Position is another CSS technique that adjusts an element's position relative to its parent. If no parent element is present, the document body is used as the parent. The syntax for absolute positioning is `position: absolute;`

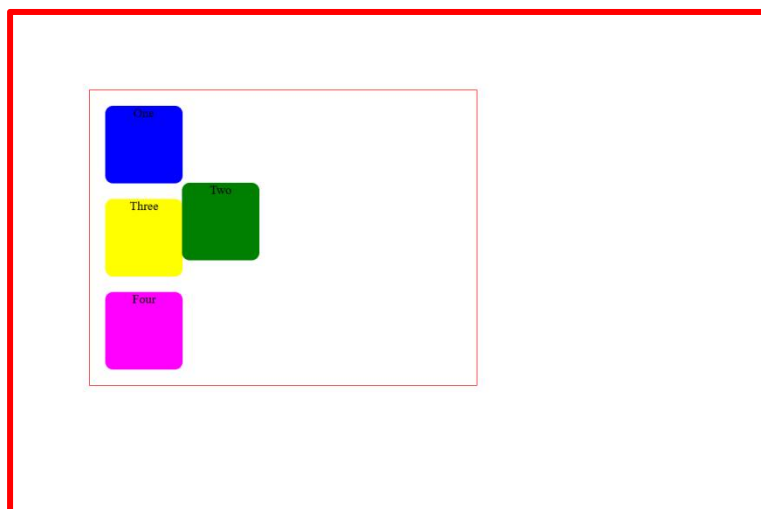
This takes an element out of the document flow and positions it relative to its nearest positioned ancestor (an ancestor with a position other than static). If no positioned ancestor exists, it's positioned relative to the initial containing block (usually the `<html>` element).

In the above example the layout of parent is default ie static, suppose we apply `position: absolute` to div class two with `left 200px` and `top 200px`, the box will move taken out of normal flow hierarchy and moved relative to body.

Example:

```
.two {  
  position: absolute;  
  top: 200px;  
  left: 200px;  
  background-color: green;  
}
```

The output will be like follows.

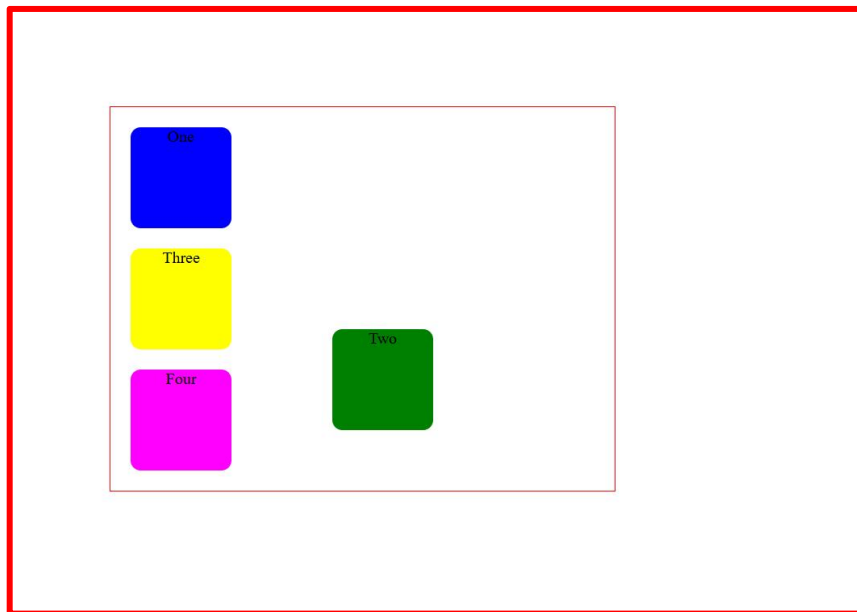


Suppose we change the position of parent to relative and for div2 same css, the div2 box will move relative to its parent.

```
.parent {  
  position: relative;  
  width: 500px;  
  border: 1px solid red;  
  text-align: center;  
}  
.two {  
  position: absolute;  
  top: 200px;  
  left: 200px;
```

```
background-color: green;  
}
```

The output will look as follows now



Fixed

This also takes an element out of the document flow but positions it relative to the viewport (the browser window). It stays in the same place even when the user scrolls.

Sticky

This is a hybrid of relative and fixed. The element is initially positioned relatively, but when it scrolls to a specified threshold, it becomes fixed

CSS Float

The CSS float property allows us to position the elements on the left or right side of the container or parent element. The element gets moved from the normal flow allowing the succeeding elements to wrap up the floated element.

Value	Description
left	Floats the element to the left side of its container.
right	Floats the element to the right side of its container.
none	Removes the float and keeps the element in the normal flow.
inherit	Inherits the float property from its parent element.

Its generally used to wrap text around images, creating component layouts etc.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>CSS float Layout</title>
    <style>
      img {
        /* floats the image to the right */
        float: right;

        width: 80px;
        height: 80px;
        border: 2px solid black;
      }

      div.parent {
        border: 2px solid black;
      }
    </style>
  </head>

  <body>
    <div class="parent">
      
      <p>
        Pokémon, short for "Pocket Monsters," is a globally beloved
        franchise that originated in Japan. It is created by
        Satoshi
        Tajiri and Ken Sugimori, and captures the imagination of
        millions with its unique blend of adventure, friendship,
        and the
        pursuit of becoming a Pokémon Master. In this fictional
        world,
        trainers embark on journeys to capture, train, and battle
        creatures known as Pokémon. Each Pokémon possesses its own
        unique abilities, characteristics, and types, allowing for
        strategic battles and endless combinations.
      </p>
    </div>
  </body>
</html>
```

Output:

Pokémon, short for "Pocket Monsters," is a globally beloved franchise that originated in Japan. It is created by Satoshi Tajiri and Ken Sugimori, and captures the imagination of millions with its unique blend of adventure, friendship, and the pursuit of becoming a Pokémon Master. In this fictional world, trainers embark on journeys to capture, train, and battle creatures known as Pokémon. Each Pokémon possesses its own unique abilities, characteristics, and types, allowing for strategic battles and endless combinations.



Example2:

```
<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8" />
    <title>CSS float Layout</title>
    <style>
      header {
        height: 80px;
        background-color: orange;
        text-align: center;
      }

      aside {
        width: 25%;

        /* floats both sidebar to left */
        float: left;
        height: 200px;
        background-color: greenyellow;
        text-align: center;
      }

      section {
        /* floats the section to the left */
        float: left;

        width: 50%;
        height: 200px;
        background-color: gold;
        text-align: center;
      }

      footer {
        /* moves the footer below the floated elements, else
        footer starts behind the floated elements */
        clear: both;

        border: 2px solid black;
        height: 40px;
        background-color: orange;
        text-align: center;
      }
    </style>
  </head>

  <body>
    <header>Header</header>

    <!-- Creating three column-based layout -->

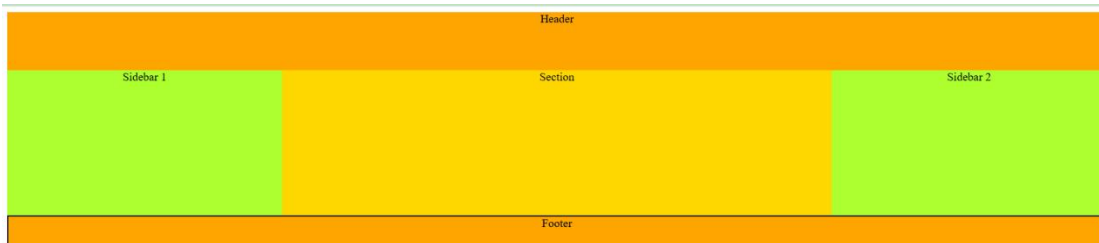
    <main>
      <aside class="left">Sidebar 1</aside>
      <section>Section</section>
      <aside class="right">Sidebar 2</aside>
    </main>

    <footer>Footer</footer>
```



```
</body>  
</html>
```

Output:



Responsive web design

Responsive web design (RWD) is a web design approach to make web pages render well on all screen sizes and resolutions while ensuring good usability. It is the way to design for a multi-device web.

Responsive Web Design is designing websites that contain flexible layouts that can scale itself according to the screen size of the device it is being viewed on. A Responsive Web Design enlarges, shrinks, resizes, or hides content present as per the screen size of the device. Using websites that look bad on the screen can be displeasing for the users and they might just close and stop using that specific website.

Benefits of Responsive web design

- ✓ **Enhanced User Experience (UX):** Ensures website is easily readable and navigable on any device. No need to pinch or zoom to read content.
- ✓ **Lower Maintenance:** Previously, businesses had to maintain two versions of their website: one for desktop and one for mobile. Responsive web design eliminates the need for a separate mobile site, reducing the time and cost associated with website maintenance.
- ✓ **Faster Loading Time:** Responsive websites tend to load faster, especially on mobile and other devices.
- ✓ **Reduced Bounce Rate:** A responsive mobile site provides a much better user experience for the visitor making them stick around for a longer duration of time and explore different areas of your site.
- ✓ **SEO Advantages:** A responsive websites is preferred from Google and other search engine.

Media Queries

CSS media query is a CSS rule that allows us to apply different styles to the webpage depending on the user's device or screen size. The styles are applied based on the characteristics of the user's device or viewport. These characteristics can include the screen width, height, orientation, and more.

The CSS @media rule enables the creation of responsive web designs.

The syntax for a media query typically includes the @ media rule followed by a condition (or set of conditions) in parentheses. Within the block, you define the CSS rules that should apply if the condition is met.

CSS Media Query Syntax

```
@media media-type and (media-feature-rule) {  
  /* CSS rules go here */  
}
```

Here,

- ✓ **media:** keyword that specifies the beginning of a media query block
- ✓ **media-type:** specifies the type of media for applying the styles; common media types include all, screen, print, and speech
- ✓ **media-feature-rule:** defines the specific conditions to apply underlying CSS styles such as width, height, or screen orientation

Media feature rules.

For screen size:

max-width, min-width

You can check individual values or combine them to create single rule.

Examples:

```
@media (max-width: 480px) {  
  /* Styles for width 480 px and less */  
}  
  
@media (min-width: 480px) {  
  /* Styles for width 480 px and more */  
}  
  
@media (min-width: 480px)and (max-width: 780px) {  
  /* Styles for width 480px to 780px */  
}
```

For screen orientation:

```
@media (orientation: portrait) {  
  /* Styles for portrait */  
}  
  
@media (orientation: landscape ) {  
  /* Styles for landscape */  
}
```

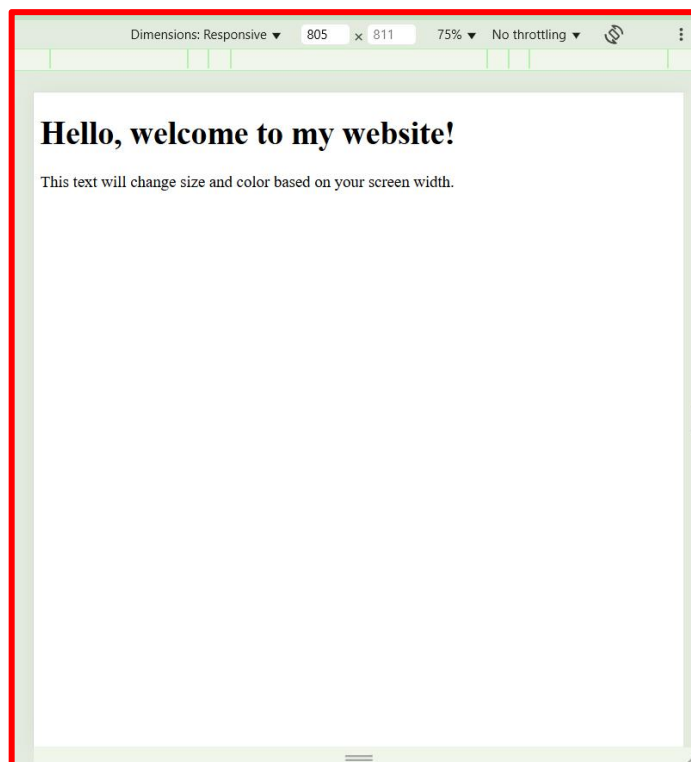
We can combine the rules for orientation and size together as well.

Adjusting font according to screen size.

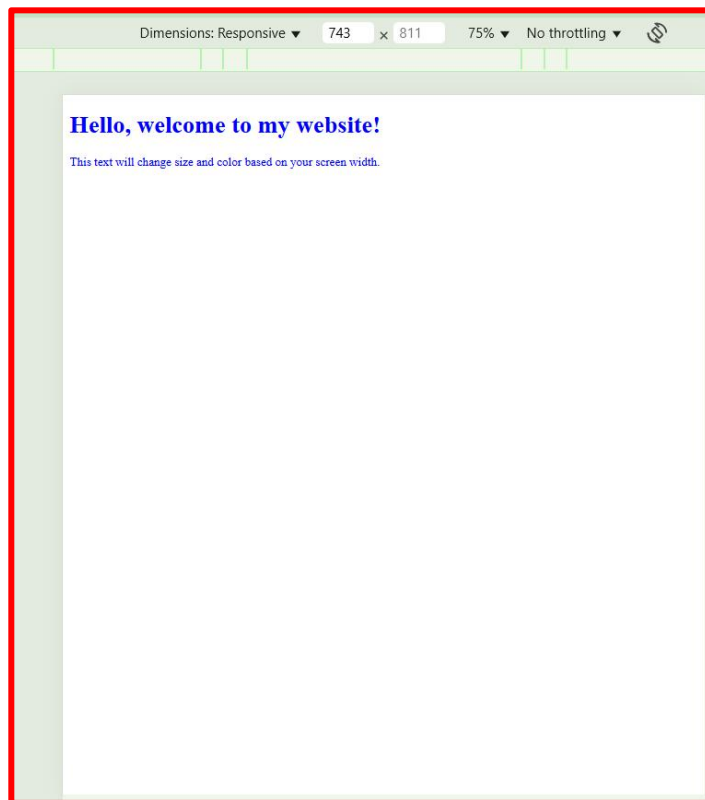
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Font Size and Color</title>
  <style>
    /* Default styles */
    body {
      font-size: 20px;
      color: black;
    }

    /* For screen widths 768px and below (e.g., tablets, mobile) */
    @media (max-width: 768px) {
      body {
        font-size: 14px;
        color: blue;
      }
    }
  </style>
</head>
<body>
  <h1>Hello, welcome to my website!</h1>
  <p>This text will change size and color based on your screen width.</p>
</body>
</html>
```

In the example above, once the device width reaches a breakpoint of 768px, the specified styles are applied, and the horizontal navigation menu transitions into a vertical menu. Here are the outputs in different widths. (You can change the size the view width on debug mode).



When screen size is reduced below 768, it will look like the following.

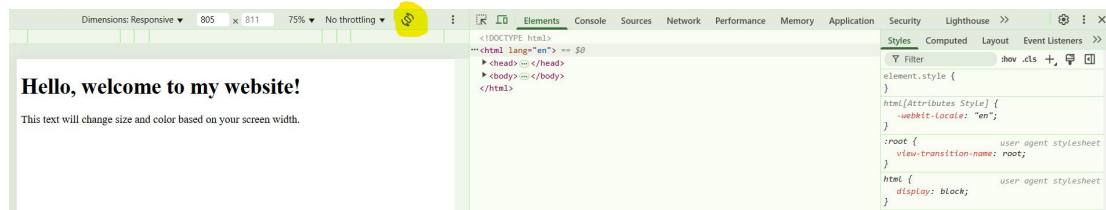


Different design for different orientation.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Font Size and Color</title>
  <style>
    /* Default styles */
    body {
      font-size: 20px;
      color: black;
    }

    /* For screen landscape orientation */
    @media screen and (orientation:landscape) {
      body {
        font-size: 14px;
        color: blue;
      }
    }
  </style>
</head>
<body>
  <h1>Hello, welcome to my website!</h1>
  <p>This text will change size and color based on your screen width.</p>
</body>
</html>
```

In this example, you can view the changes in UI according to orientation change. You can test by going to debug mode and by changing the screen orientation.



Navigation bar adjusting according to screen size

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport"
content="width=device-width, initial-scale=1">
<style>
* {
box-sizing: border-box;
}
/* Style the top navigation bar */
.topnav {
overflow: hidden;
background-color: #333;
}
/* Style the topnav links */
.topnav a {
float: left;
display: block;
color: #f2f2f2;
text-align: center;
padding: 14px 16px;
text-decoration: none;
}
/* On screens that are 600px wide or less, make the
menu links stack on top of each other instead of next
to each other */
@media screen and (max-width: 600px) {
.topnav a {
float: none;
width: 100%;
}
}
</style>
</head>
<body>
<h2>Responsive navigation menu</h2>
<div class="topnav">
<a href="#">Nav1</a>
<a href="#">Nav2</a>
<a href="#">Nav3</a>
</div>
</body>
</html>
```

Bootstrap

Bootstrap is a free and open-source tool collection for creating responsive websites and web applications. It is the most popular HTML, CSS, and JavaScript framework for developing responsive, mobile-first websites.

Bootstrap is a free front-end framework that is popular among developers today, particularly those who work in the web design world. It's easy to use and saves developers a lot of time from having to write HTML, CSS, and JavaScript code manually.

Features of Bootstrap

- ✓ **Responsive Design:** Bootstrap's grid system and components adapt seamlessly to different screen sizes, from desktops to tablets to smartphones.
- ✓ **Pre-styled Components:** Buttons, forms, navigation bars, alerts, and many other common UI elements are styled out-of-the-box, saving you development time.
- ✓ **Grid System:** Bootstrap's powerful grid system makes it easy to create complex layouts with rows and columns.
- ✓ **JavaScript Plugins:** Interactive elements like modals, carousels, and dropdowns are powered by JavaScript plugins.
- ✓ **Customization:** While Bootstrap provides default styles, it's highly customizable through Sass variables, allowing you to tailor the look and feel to your project.
- ✓ **Large Community:** A vast community provides ample support, documentation, and resources.

Installation of Bootstrap

We can install bootstrap in different ways.

1. **CDN (Content Delivery Network):** The easiest way is to use Bootstrap's CDN.

Include the following links in your HTML <head> section:

```
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-T3c6CoIi6uLrA9TneNEoa7rxnatzjcDSCmG1MXxSR1GAsXEV/Dwwykc2MPK8M2HN"
crossorigin="anonymous">
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js" integrity="sha384-C6RzsynM9kWDrmNeT87bh95OGNyZPhcTNXj1NW7RuBCsyN/o0jlpcV8Qyq46cDfL"
crossorigin="anonymous"></script>
```

2. **Download:** You can download Bootstrap's compiled CSS and JavaScript files from getbootstrap.com. Then, include the files in your project and link to them in your HTML.

```
<link rel="stylesheet" href="path/to/bootstrap.min.css">
```

3. **Package Managers (npm, yarn):** If you're using a package manager like npm or yarn, you can install Bootstrap:

```
npm install bootstrap  
# or  
yarn add bootstrap
```

Then, you'll need to include the necessary files in your project (often through a build process).

Explore Examples on your own